

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа

«Информационная система
управления фильмами»

по курсу «Информационные системы»

Вариант 53434963

Выполнил:

Якименко Владислав Игоревич, Р3313

Проверил:

Кобик Никита Алексеевич

Содержание

1	Цель работы и текст задания	1
2	UML диаграмма классов	5
3	UML диаграмма пакетов	6
4	Исходный код	7
5	Выводы	7

1 Цель работы и текст задания

Цель работы — разработать многопользовательскую информационную систему для управления фильмами и связанными объектами с использованием Jakarta EE, CDI, JPA/Hibernate и PostgreSQL. Ниже приведён текст индивидуального задания для варианта 53434963.

Реализовать информационную систему, которая позволяет взаимодействовать с объектами класса Movie, описание которого приведено ниже:

```
public class Movie {
    private long id; //Значение поля должно быть больше 0, Значение этого поля должно
    быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDate creationDate; //Поле не может быть null, Значение этого
    поля должно генерироваться автоматически
    private Long oscarsCount; //Значение поля должно быть больше 0, Поле может быть null
    private double budget; //Значение поля должно быть больше 0
    private float totalBoxOffice; //Значение поля должно быть больше 0
    private Mpaarating mpaarating; //Поле не может быть null
    private Person director; //Поле не может быть null
    private Person screenwriter;
    private Person operator; //Поле может быть null
    private Integer length; //Поле не может быть null, Значение поля должно быть больше 0
    private Long goldenPalmCount; //Значение поля должно быть больше 0, Поле может быть
    null
    private String tagline; //Поле может быть null
    private MovieGenre genre; //Поле может быть null
}

public class Coordinates {
    private Integer x; //Поле не может быть null
    private Float y; //Максимальное значение поля: 2, Поле не может быть null
}

public class Person {
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Color eyeColor; //Поле не может быть null
    private Color hairColor; //Поле не может быть null
    private Location location; //Поле может быть null
    private Double height; //Поле не может быть null, Значение поля должно быть больше 0
    private Country nationality; //Поле может быть null
}

public class Location {
    private Integer x; //Поле не может быть null
    private Integer y; //Поле не может быть null
    private float z;
    private String name; //Поле не может быть null
}

public enum Mpaarating {
    G,
    PG,
```

```

    R,
    NC_17;
}
public enum MovieGenre {
    ACTION,
    WESTERN,
    COMEDY,
    FANTASY;
}
public enum Color {
    RED,
    YELLOW,
    WHITE;
}
public enum Country {
    UNITED_KINGDOM,
    GERMANY,
    THAILAND,
    SOUTH_KOREA;
}

```

Разработанная система должна удовлетворять следующим требованиям:

- Основное назначение информационной системы - управление объектами, созданными на основе заданного в варианте класса.
- Необходимо, чтобы с помощью системы можно было выполнить следующие операции с объектами: создание нового объекта, получение информации об объекте по ИД, обновление объекта (модификация его атрибутов), удаление объекта. Операции должны осуществляться в отдельных окнах (интерфейсах) приложения. При получении информации об объекте класса должна также выводиться информация о связанных с ним объектах.
- При создании объекта класса необходимо дать пользователю возможность связать новый объект с объектами вспомогательных классов, которые могут быть связаны с созданным объектом и уже есть в системе.
- Выполнение операций по управлению объектами должно осуществляться на серверной части (не на клиенте), изменения должны синхронизироваться с базой данных.
- На главном экране системы должен выводиться список текущих объектов в виде таблицы (каждый атрибут объекта - отдельная колонка в таблице). При отображении таблицы должна использоваться пагинация (если все объекты не помещаются на одном экране).
- Нужно обеспечить возможность фильтровать/сортировать строки таблицы, которые показывают объекты (по значениям любой из строковых колонок). Фильтрация элементов должна производиться по неполному совпадению.

- Переход к обновлению (модификации) объекта должен быть возможен из таблицы с общим списком объектов и из области с визуализацией объекта (при ее реализации).
- При добавлении/удалении/изменении объекта, он должен автоматически появиться/исчезнуть/измениться в интерфейсах у других пользователей, авторизованных в системе.
- Если при удалении объекта с ним связан другой объект, связанные объекты должны удаляться.
- Пользователи должны иметь возможность просмотра всех объектов. Для модификации объекта должно открываться отдельное диалоговое окно. При вводе некорректных значений в поля объекта должны появляться информативные сообщения о соответствующих ошибках.

В системе должен быть реализован отдельный пользовательский интерфейс для выполнения специальных операций над объектами:

- Удалить один (любой) объект, значение поля tagline которого эквивалентно заданному.
- Вернуть массив объектов, значение поля tagline которых содержит заданную подстроку.
- Вернуть массив объектов, значение поля genre которых меньше заданного.
- Получить список сценаристов, ни один фильм которых не получил ни одного "Оскара".
- Равномерно перераспределить все "Оскары" полученные фильмами одного жанра, между фильмами другого жанра.
- Представленные операции должны быть реализованы в качестве функций БД, которые необходимо вызывать из уровня бизнес-логики приложения.

Особенности хранения объектов, которые должны быть реализованы в системе:

- Организовать хранение данных об объектах в реляционной СУБД (PostgreSQL). Каждый объект, с которым работает ИС, должен быть сохранен в базе данных.
- Все требования к полям класса (указанные в виде комментариев к описанию классов) должны быть выполнены на уровне ORM и БД.
- Для генерации поля id использовать средства базы данных.
- Для подключения к БД на кафедральном сервере использовать хост pg, имя базы данных - studs, имя пользователя/пароль совпадают с таковыми для подключения к серверу.

При создании системы нужно учитывать следующие особенности организации взаимодействия с пользователем:

- Система должна реагировать на некорректный пользовательский ввод, ограничивая ввод недопустимых значений и информируя пользователей о причине ошибки.
- Переходы между различными логически обособленными частями системы должны осуществляться с помощью меню.
- При добавлении/удалении/изменении объекта, он должен автоматически появиться/исчезнуть/измениться на области у всех других клиентов.

При разработке ИС должны учитываться следующие требования:

- В качестве основы для реализации ИС необходимо использовать Jakarta EE, CDI beans.
- Для создания уровня хранения необходимо использовать JPA + Hibernate.
- Разные уровни приложения должны быть отделены друг от друга, разные логические части ИС должны находиться в отдельных компонентах.

2 UML диаграмма классов

Показаны предметные сущности, поля и кратности связей. Поля id вспомогательных сущностей и version добавлены для хранения ссылок и контроля конкурирующих изменений. Доступ к приватным полям предоставляется методами, которые для компактности диаграммы не показаны.

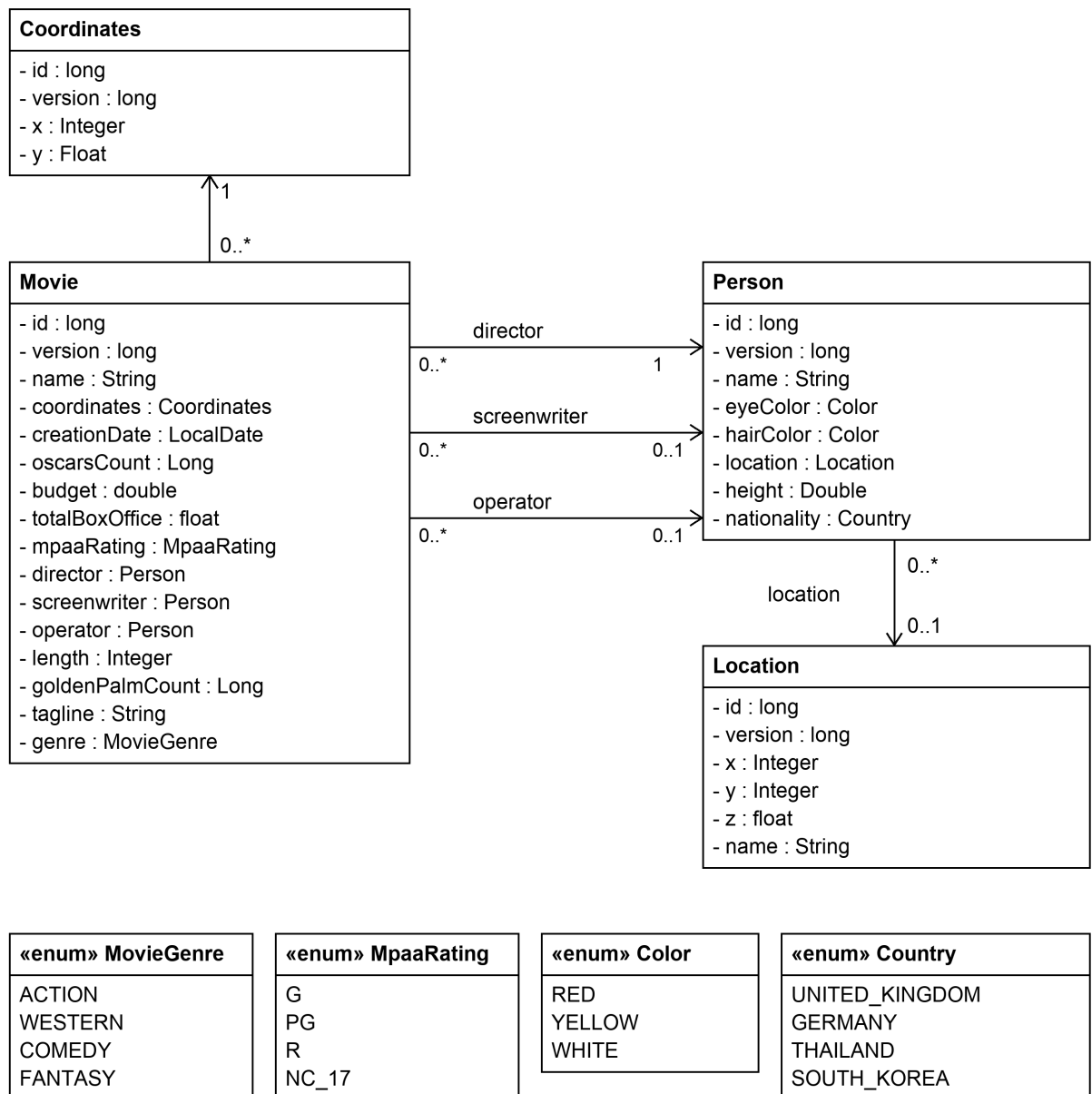


Рис. 1: Классы предметной области

3 UML диаграмма пакетов

Диаграмма показывает основные направления вызовов и использования моделей. Вспомогательные зависимости обработки ошибок не показаны. Интерфейс браузера обращается к REST-ресурсам пакета web.

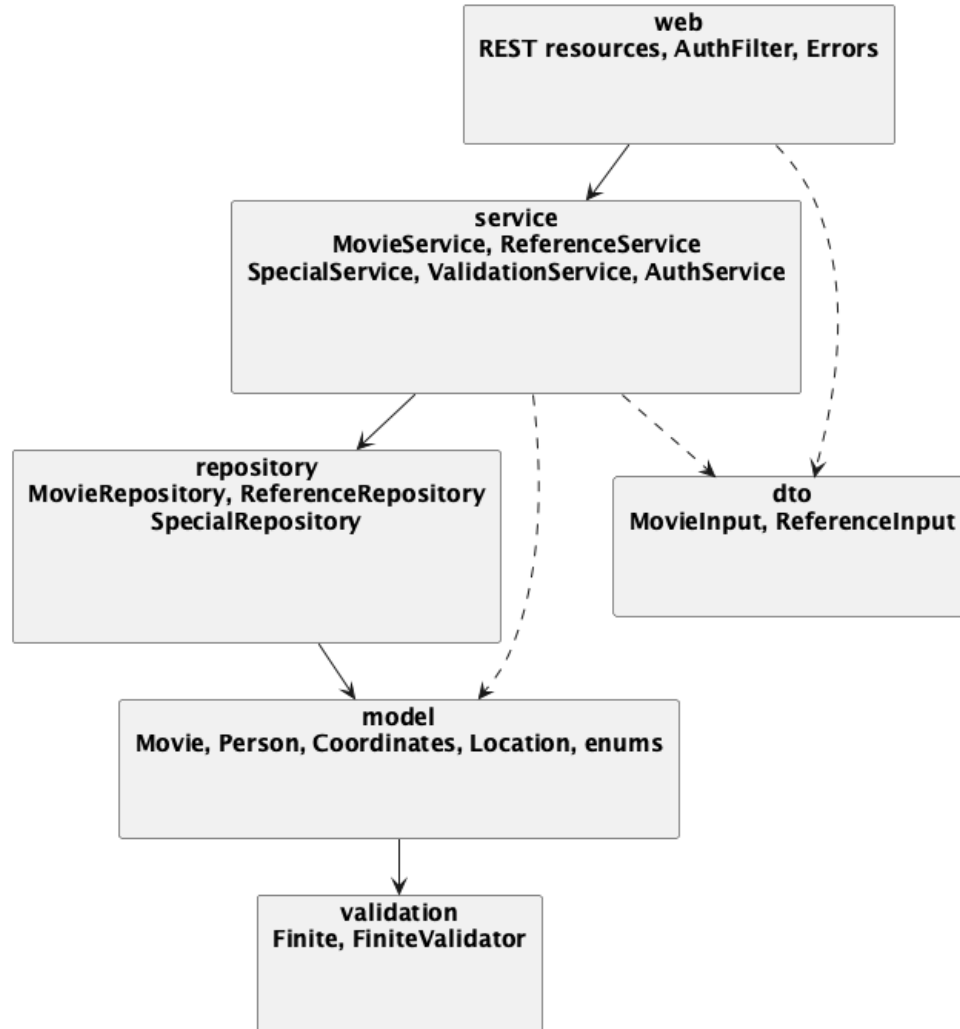


Рис. 2: Пакеты приложения

4 Исходный код

Исходный код приложения доступен в публичном репозитории: <https://github.com/delovoyhomie/movie-lab-53434963>.

Серверная часть находится в `src/main/java`, интерфейс — в `src/main/webapp`, схема базы данных и SQL-функции — в каталоге `db`.

5 Выводы

Реализована информационная система управления фильмами с разделением HTTP-интерфейса, бизнес-логики и доступа к данным. JPA и Hibernate связывают предметные сущности с PostgreSQL, а ограничения Java и базы данных защищают целостность данных независимо от клиентского интерфейса.

Специальные операции выполняются функциями PostgreSQL. Транзакции обеспечивают атомарность перераспределения наград; версии сущностей предотвращают сохранение устаревших форм. Периодический опрос версии модели синхронизирует интерфейсы пользователей. Для небольшого учебного приложения выбраны простая архитектура одного WAR и минимальный интерфейс без дополнительных фреймворков.